Aula de hoje: início do TCC em Python 3

24/10/2026 · Turma de sábado, 10h30 · Instrutor Pedro Leonardo

O que vamos fazer hoje

Terminamos a lista de revisão de Python 3. Agora começa o TCC: o projeto final do ano, onde cada aluno cria o **seu próprio programa**, que pode ser um **jogo** ou um **sistema**, vamos iniciar desde a ideia inicial até ele funcionar do começo ao fim.

O TCC vai passar por estas fases. Hoje começamos pelas três primeiras:

Data de entrega: 07/12/2026. O último sábado de aula antes da entrega é **05/12**, então o projeto precisa estar pronto e testado até essa aula.

O que é o TCC

O TCC (Trabalho de Conclusão de Curso) é o projeto final do ano, no qual você coloca em prática os conhecimentos adquiridos ao longo das aulas. Você cria o seu próprio projeto: a ideia, o funcionamento, a programação e a forma como o usuário vai interagir com ele.

O objetivo principal é estimular a **autonomia**, a **criatividade** e a **resolução de problemas**. As decisões são suas. O instrutor acompanha o projeto, dá orientações e ajuda quando você travar, mas vai sempre incentivar você a encontrar a solução primeiro.

No final, o projeto deve mostrar a sua evolução e as habilidades que você ganhou durante o curso. Nesta turma o TCC é feito em **Python 3**, rodando no terminal (só texto).

Etapa 1: os requisitos mínimos

Por quê: o requisito mínimo é a linha de chegada mínima. Ele garante que o projeto use os conteúdos principais do curso. Fazer mais do que o mínimo é sempre bem-vindo, fazer menos não é aceito.

Estrutura do código

Requisito	O que significa	Onde vimos
Aproximadamente 200 linhas	O projeto tem que ter conteúdo suficiente para ser usado por alguns minutos. É uma referência de tamanho, não uma meta para encher com linhas repetidas.	Todo o curso
Uso do `if`	O programa precisa tomar decisões de acordo com o que o usuário faz.	Exercícios 5 a 8
Uso do `while`	O programa precisa repetir ações, como um menu que aparece de novo depois de cada escolha.	Exercícios 9 a 12
Pelo menos 2 laços de repetição	Dois laços diferentes no código, por exemplo o laço do menu principal e o laço de uma das opções.	Exercícios 9 a 14

`while` dentro de `while`
ou
`if`, `elif` e `else`
	Pelo menos uma das duas estruturas: um laço `while` dentro de outro (ex: um carrinho de compras dentro do menu), ou uma decisão com `if`, `elif` e `else` (ex: um menu com várias opções).	Exercícios 7, 13, 14 e 15

Variáveis e entradas

Requisito	O que significa	Onde vimos
Pelo menos 2 tipos de variáveis	Usar tipos diferentes, por exemplo: texto (`str`), número inteiro (`int`), número com vírgula (`float`) ou verdadeiro/falso (`bool`).	Exercícios 3 e 4
Variáveis que fazem sentido	O nome mostra o que a variável guarda: `saldo`, `vida`, `pontos`. Nada de `x`, `a1` ou `coisa`.	Todo o curso
Mais de um tipo de input	O usuário precisa responder coisas diferentes, por exemplo um texto (o nome) e um número (uma quantidade com `int(input())`).	Exercícios 3, 4 e 10

Qualidade do projeto

Requisito	O que significa
Sistema com finalidade e propósito	O programa precisa servir para alguma coisa: ter um objetivo claro, que dá para explicar em uma frase.
Textos bem formatados	As mensagens são fáceis de ler: títulos, separadores, opções numeradas, sem erros de português e sem texto todo grudado.
Comentários no código	Cada variável e cada parte importante do código têm um comentário curto explicando para que servem.

Lembrando a regra de sempre: não complique mais do que o necessário. Um projeto simples, que funciona do começo ao fim, vale mais do que um projeto grande que trava no meio.

Etapa 2: escolher a ideia

Por quê: o TCC pode ser um jogo ou um sistema. Em Python, sem imagens, tudo é feito com texto, então o segredo é escolher uma ideia que combine com texto e com as ferramentas que você já sabe usar.

Sugestões de projeto:

Projeto	Como funciona	Estruturas que ele usa
Sistema de mercado	O usuário escolhe produtos, coloca no carrinho, vê o total e paga com o saldo que tem.	`while` do carrinho dentro do `while` do menu

Jogo de RPG	Um herói com vida, ataque e moedas enfrenta inimigos em turnos: atacar, defender, usar item ou fugir.	`while` da batalha dentro do `while` do jogo
Aventura narrativa	O usuário lê uma situação e escolhe o que fazer. Cada escolha leva a um caminho e a um final diferente.	`if`, `elif` e `else` em sequência
Calculadora	Uma calculadora com menu de operações, histórico de contas, ou cálculos prontos (média de notas, IMC, desconto).	Menu com `while` e `if`, `elif` e `else`

Como escolher:

- Escolha uma ideia que **você** gostaria de usar. Você vai passar mais de um mês com esse projeto.
- Confira se a ideia tem uma **finalidade**: quem vai usar e para quê? Se não dá para responder em uma frase, a ideia ainda está vaga.
- Pode misturar ideias: um RPG com mercado entre as batalhas, ou uma calculadora com várias ferramentas.
- Fuja de ideias que precisam de imagem ou de tempo real. No terminal, o usuário sempre responde e depois o programa continua.

Etapa 3: planejar a estrutura no bloco de notas

Por quê: programar sem saber o que o projeto vai ser é o jeito mais rápido de ficar perdido. Com a estrutura escrita, você sabe o que falta fazer em cada aula.

Como fazer:

- Abra o Bloco de Notas e crie um arquivo com o nome do projeto, por exemplo `mercado_plano.txt`.
- Escreva a finalidade do projeto em uma frase.
- Liste as opções do menu: o que o usuário pode fazer.
- Liste as variáveis, o tipo de cada uma e para que ela serve.
- Marque onde cada requisito vai aparecer: onde fica cada laço, onde fica o `if` e quais são os inputs.
- Atualize o arquivo sempre que mudar alguma coisa no projeto.

Exemplo de plano:

```
PROJETO: Mercadinho do Bairro
FINALIDADE: simular as compras de um cliente com saldo limitado

MENU:
  1 - Comprar produtos
  2 - Ver carrinho
  3 - Pagar
  4 - Sair

VARIÁVEIS:
  cliente  (texto)       nome do cliente
  saldo    (inteiro)     dinheiro para gastar
  total    (inteiro)     valor do carrinho
  itens    (inteiro)     quantidade de itens no carrinho
  aberto   (verd/falso)  se o mercado continua aberto

REQUISITOS:
  laço 1: while do menu principal
  laço 2: while da compra, dentro do laço 1
  if/elif/else: menu e escolha do produto
  inputs: nome (texto) e quantidade (número)
```

Etapa 4: dicas para um programa bom de usar

Por quê: em um programa de terminal, o texto é tudo o que o usuário vê. Se ele estiver confuso, o usuário se perde; se estiver claro e organizado, o programa fica agradável de usar.

O usuário sempre precisa saber o que pode fazer

- Mostre as opções numeradas e sempre do mesmo jeito. O usuário não deve adivinhar o que pode digitar.
- Mostre as informações importantes antes de cada escolha (saldo, total, pontos, vida).
- Quando o usuário digitar algo errado, avise e mostre as opções de novo.

Toda ação precisa de uma resposta

- Se o usuário adicionou um produto, mostre o novo total. Se fez uma conta, mostre o resultado com uma frase, não só o número.
- Mensagens curtas e diretas. Uma tela cheia de texto faz o usuário pular a leitura.

Organize o texto na tela

- Use separadores (`"=" * 30`, `"-" * 30`) para dividir títulos, menus e resultados.
- Use `print()` vazio para dar espaço entre uma parte e outra.
- Revise o português: um erro de digitação no menu passa a impressão de projeto mal feito.

Teste com um colega

- Fique quieto enquanto ele usa. Se você precisar explicar alguma coisa, o texto do programa precisa mudar.
- Anote onde ele travou, o que digitou de errado e o que não entendeu.

Testar e problemas comuns

Antes de entregar, rode o projeto e confira:

- O projeto tem aproximadamente 200 linhas de código
- Tem uma finalidade clara, que dá para explicar em uma frase
- Usa `if` e `while`
- Tem pelo menos 2 laços de repetição
- Usa `while` dentro de `while`, ou `if`, `elif` e `else`
- Usa pelo menos 2 tipos de variáveis, todas com nomes que fazem sentido
- Pede mais de um tipo de input (ex: texto e número)
- Toda variável e toda parte importante tem um comentário explicando para que serve
- Os textos estão formatados e sem erros de português
- Dá para usar do começo ao fim sem nenhum erro aparecer
- Existe uma opção para sair
- O arquivo está salvo, com uma cópia guardada em outro lugar

Próximas aulas

- **Primeira função principal**: desenvolvimento da opção mais importante do projeto.
- **Acompanhamento individual** dos projetos.
- **Teste com colega** e correção de bugs.

Tarefa em aula

- **Escolha a ideia do seu projeto (jogo ou sistema)**
- **Escreva o plano no bloco de notas: finalidade, menu, variáveis e onde fica cada requisito**
- **Crie o arquivo do projeto e salve com o nome dele**